

Introduction to Parallel Algorithms and Programming

TD1: Parallel algorithms in the PRAM model

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

- The exercises that we cannot cover in class are homework: solutions will be posted, and we may go over some of them later in class.
- The algorithms of an exercise may be used in the following exercises.
- Arrays are indexed from 1; NIL denotes the null pointer; cx denotes a copy of the array x .
- The **time** of an algorithm is its parallel running time on the PRAM, using the number of PUs it is given; **work**, **cost**, work-optimality and efficiency are defined as in the lecture, with respect to T_{seq} .
- Building blocks from the lecture, all on an EREW PRAM with N PUs:

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Exercise 1: Is the array sorted?

Let $A[1 \dots N]$ be an array of N integers.

- a) Design an algorithm $\text{IS_SORTED}(A[1 \dots N])$ that returns *true* if $A[1] \leq A[2] \leq \dots \leq A[N]$ and *false* otherwise, in $O(1)$ time on a CRCW PRAM with N PUs. List all the concurrency modes that can be employed for the writes of this algorithm.
- b) Verify the reads of your algorithm, by decomposing them into elementary steps as was done for list ranking in the lecture: are they exclusive? Does the algorithm need concurrent reads, or only concurrent writes?
- c) Design an EREW PRAM version of IS_SORTED with N PUs. What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 1(a): ISORTED on a CRCW PRAM

- One PU per pair of consecutive elements compares them, and reports a violation into a shared flag *sorted*.
- Line 3: several PUs may write *sorted* at once, but they all write the same value *false*: every mode works.
 - **Consistent** mode: enough, all the written values are equal.
 - **Arbitrary** and **priority** modes: same result, whichever PU wins.
 - **Fusion** mode: every PU writes its own verdict $sorted \leftarrow (A[i] \leq A[i + 1])$ instead, with **and** as the operation (or **min**, with *false* = 0, *true* = 1).
- $T(N) = O(1)$ with $N - 1$ PUs: line 3 is a single comparison.
- $W(N) = O(N)$, $C(N) = O(N)$, and $T_{seq}(N) = O(N)$: **work-optimal** and **efficient**.

ISORTED($A[1 \dots N]$)

```

1: sorted  $\leftarrow$  true
2: forall  $i = 1, \dots, N - 1$  in parallel do
3:   if  $A[i] > A[i + 1]$  then sorted  $\leftarrow$  false
4: return sorted

```

TD1 - PRAM

Let $A[1 \dots N]$ be an array of N integers.

- a) Design an algorithm $\text{ISORTED}(A[1 \dots N])$ that returns *true* if $A[1] \leq A[2] \leq \dots \leq A[N]$ and *false* otherwise, in $O(1)$ time on a CRCW PRAM with N PUs. List all the concurrency modes that can be employed for the writes of this algorithm.
- b) Verify the reads of your algorithm, by decomposing them into elementary steps as was done for list ranking in the lecture: are they exclusive? Does the algorithm need concurrent reads, or only concurrent writes?
- c) Design an EREW PRAM version of ISORTED with N PUs. What are its time, work and cost? Is it work-optimal? Is it efficient?

- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring**
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Let $next[1 \dots N]$ be a linked list of N nodes: $next[i]$ is the successor of node i , and $next[\ell] = \text{NIL}$ for the last node ℓ . The head h of the list is the only node that no other node points to. We want to close the list into a ring, by making the last node point to the head: $next[\ell] \leftarrow h$.

- a) Design an EREW PRAM algorithm $\text{LISTTORING}(next[1 \dots N])$ with N PUs that updates $next[1 \dots N]$ into a ring and returns the index h of the head. Verify that all reads and writes are exclusive.
- b) What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 2(a): LISTToRING

- The head is the only node that is not pointed to: every node **marks its successor**, and the unmarked node is the head.
- Line 4: two PUs never write the same cell $mark[next[i]]$, because $next$ is **injective** (two nodes have different successors).
- Line 6: exactly one PU writes h (the head is unique).
- Line 8: exactly one PU reads h (the last node is unique) and writes its own cell.
- All the other accesses are on the cells of the PU itself: **EREW**.
- h is written in one parallel loop and read in *another* one (lines 6 and 8): a write and a read of the same cell in the same loop would conflict.

LISTTORING($next[1 \dots N]$)

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $mark[i] \leftarrow 0$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:   if  $next[i] \neq NIL$  then  $mark[next[i]] \leftarrow 1$ 
5: forall  $i = 1, \dots, N$  in parallel do
6:   if  $mark[i] = 0$  then  $h \leftarrow i$ 
7: forall  $i = 1, \dots, N$  in parallel do
8:   if  $next[i] = NIL$  then  $next[i] \leftarrow h$ 
9: return  $h$ 

```


- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list**
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Exercise 3: Inverting a list

Let $next[1 \dots N]$ be a linked list of N nodes.

- a) Design an EREW PRAM algorithm $\text{INVERT}(\text{next}[1 \dots N])$ with N PUs that computes the inverse of this linked list, and returns it in a pointer array $\text{prev}[1 \dots N]$ (e.g., if $\text{next}[3] = 7$, then $\text{prev}[7] = 3$).
- b) What are its time, work and cost? Is it work-optimal? Is it efficient?
- c) Now let $\text{next}[1 \dots N]$ be a ring (no pointer is NIL), and h the index of one of its nodes. Design an EREW PRAM algorithm $\text{RINGTOLIST}(\text{next}[1 \dots N], h)$ with N PUs that turns the ring into a list whose head is h . What is its time?

Exercise 3(a): INVERT

- Every node writes itself into the *prev* cell of its successor: $prev[next[i]] \leftarrow i$.
- Line 4: the writes are exclusive because *next* is **injective**: two different nodes have two different successors, hence write two different cells.
- Line 2 gives $prev[h] = \text{NIL}$ for the head *h*, the only node that nobody writes (and the only node with $next[i] = \text{NIL}$, the last one, writes nothing).
- The reads are on the cells of PU *i* only: **EREW**.
- INVERT is the tool for traversing a list *in the other direction*: used in (c), and again in exercises 6 and 12.

INVERT(*next*[1...*N*])

```
1: forall  $i = 1, \dots, N$  in parallel do
2:    $prev[i] \leftarrow \text{NIL}$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:   if  $next[i] \neq \text{NIL}$  then  $prev[next[i]] \leftarrow i$ 
5: return  $prev[1 \dots N]$ 
```

- Two parallel loops of $O(1)$ time: $T(N) = O(1)$ with N PUs.
- N PUs created in each loop, $O(1)$ work each: $W(N) = O(N)$; $C(N) = O(N)$.
- Sequentially, every pointer must be read once: $T_{seq}(N) = O(N)$.
- Work-optimal** and **efficient**.

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $prev[i] \leftarrow \text{NIL}$ 
3: forall  $i = 1, \dots, N$  in parallel do
4:   if  $next[i] \neq \text{NIL}$  then  $prev[next[i]] \leftarrow i$ 
5: return  $prev[1 \dots N]$ 

```

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ≡ ▶ ◀ ≡ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻

Exercise 3: Inverting a list

Let $next[1 \dots N]$ be a linked list of N nodes.

- a) Design an EREW PRAM algorithm $\text{INVERT}(\text{next}[1 \dots N])$ with N PUs that computes the inverse of this linked list, and returns it in a pointer array $\text{prev}[1 \dots N]$ (e.g., if $\text{next}[3] = 7$, then $\text{prev}[7] = 3$).
- b) What are its time, work and cost? Is it work-optimal? Is it efficient?
- c) Now let $\text{next}[1 \dots N]$ be a ring (no pointer is NIL), and h the index of one of its nodes. Design an EREW PRAM algorithm $\text{RINGTOLIST}(\text{next}[1 \dots N], h)$ with N PUs that turns the ring into a list whose head is h . What is its time?

Exercise 3(c): RINGToLIST

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

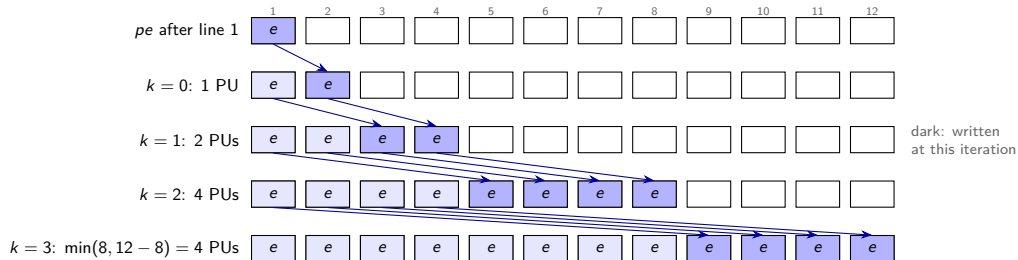
- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction**
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

- a) The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name BCAST(e, N), so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- b) Write a *recursive* version BCASTREC(e, N) of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- c) Design BCASTP(e, N, P) that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- d) Write an *iterative* version REDUCE($A[1 \dots N], \otimes$) of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- e) The reduction of the lecture is *recursive*: write it as REDUCEREC($A[1 \dots N], \otimes$). Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- f) Design REDUCEP($A[1 \dots N], \otimes, P$) that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

Exercise 4(a): a work-optimal broadcast on $N = 12$ cells

Before iteration k , the cells $pe[1 \dots 2^k]$ hold e ; the iteration copies them onto the next 2^k cells. In the lecture, N PUs are created and the **if** keeps all but 2^k of them idle: create only the PUs that copy, $\min(2^k, N - 2^k)$ of them (the min clips the last iteration to the cells that exist).



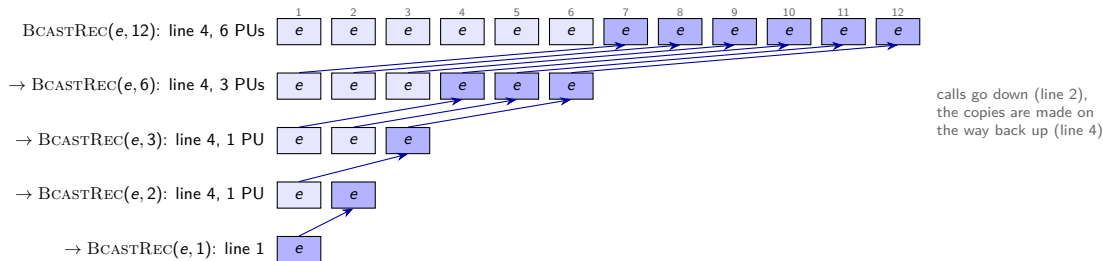
Exercise 4: Efficient broadcast and reduction

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

- a) The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name BCAST(e, N), so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- b) Write a *recursive* version BCASTREC(e, N) of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- c) Design BCASTP(e, N, P) that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- d) Write an *iterative* version REDUCE($A[1 \dots N], \otimes$) of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- e) The reduction of the lecture is *recursive*: write it as REDUCEREC($A[1 \dots N], \otimes$). Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- f) Design REDUCEP($A[1 \dots N], \otimes, P$) that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

Exercise 4(b): a recursive broadcast on $N = 12$ cells

Fill the first half of the array by a recursive call, then copy it onto the second half in one parallel step. The calls go down to size 1 before the first copy (sizes 12, 6, 3, 2, 1); the copies are made on the way back up. One row per call, each row being $pe[1 \dots n]$ as the call returns it.



Exercise 4(b): BCASTREC, the recursive broadcast

- The **first half** by the recursive call (line 2), then one parallel step copies it onto the **second half** (lines 3–4): $\lfloor n/2 \rfloor$ PUs, one per cell to write.
- Accesses: PU i reads $pe[i]$, $i \leq \lfloor n/2 \rfloor \leq \lceil n/2 \rceil$, and writes $pe[\lceil n/2 \rceil + i]$, beyond $\lceil n/2 \rceil$: disjoint cells, distinct PUs: **exclusive**.
- When N is a power of two, (a) and (b) make exactly the same copies at the same steps; otherwise the intermediate sizes differ: 1, 2, 4, 8, 12 for (a), 1, 2, 3, 6, 12 for (b), in $\lceil \log_2 N \rceil = 4$ steps either way.

$$\text{BCASTREC}(e, n)$$

```

1: if  $n = 1$  then  $pe[1] \leftarrow e$ ; return  $pe[1 \dots 1]$ 
2:  $pe[1 \dots \lceil n/2 \rceil] \leftarrow \text{BCASTREC}(e, \lceil n/2 \rceil)$ 
3: forall  $i = 1, \dots, \lfloor n/2 \rfloor$  in parallel do
4:    $pe[\lceil n/2 \rceil + i] \leftarrow pe[i]$ 
5: return  $pe[1 \dots n]$ 

```

Exercise 4(b): BCASTREC, work-optimal, but efficient?

- Time: $T(n) = T(\lceil n/2 \rceil) + O(1)$, hence $T(N) = O(\log N)$, with at most $\lfloor n/2 \rfloor \leq N/2$ PUs created at any level.
- Work: $W(n) = W(\lceil n/2 \rceil) + \lfloor n/2 \rfloor + O(1)$; every cell $pe[2 \dots N]$ is written **exactly once**: $W(N) = N - 1 + O(\log N) = O(N)$, **work-optimal**.
- Efficient? **No**, like (a): $C(N) = O(N \log N)$ on a machine with N PUs. Iterative or recursive, the PUs that are not created are paid for.

BCASTREC(e, n)

```

1: if  $n = 1$  then  $pe[1] \leftarrow e$ ; return  $pe[1 \dots 1]$ 
2:  $pe[1 \dots \lceil n/2 \rceil] \leftarrow \text{BCASTREC}(e, \lceil n/2 \rceil)$ 
3: forall  $i = 1, \dots, \lfloor n/2 \rfloor$  in parallel do
4:    $pe[\lceil n/2 \rceil + i] \leftarrow pe[i]$ 
5: return  $pe[1 \dots n]$ 

```

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
BCAST (EREW)	N	$O(N)$	$O(\log N)$	$O(N)$	yes	$O(N \log N)$	no
BCASTREC (EREW)	N	$O(N)$	$O(\log N)$	$O(N)$	yes	$O(N \log N)$	no

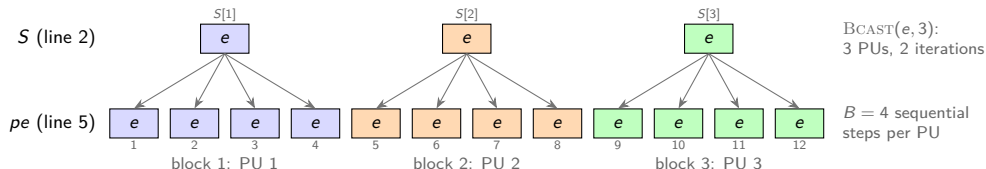
Exercise 4: Efficient broadcast and reduction

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{\text{seq}}(N) = O(N)$ for both problems.

- The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name $\text{BCAST}(e, N)$, so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- Write a *recursive* version $\text{BCASTREC}(e, N)$ of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- Design $\text{BCASTP}(e, N, P)$ that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- Write an *iterative* version $\text{REDUCE}(A[1 \dots N], \otimes)$ of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- The reduction of the lecture is *recursive*: write it as $\text{REDUCEREC}(A[1 \dots N], \otimes)$. Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- Design $\text{REDUCEP}(A[1 \dots N], \otimes, P)$ that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

Exercise 4(c): a broadcast with $P = 3$ PUs on $N = 12$ cells

Cut the array into P blocks of $B = N/P$ consecutive cells. The parallel part first, on P cells: broadcast e to $S[1 \dots P]$ with (a) and P PUs; then the sequential part, on the blocks: PU j copies $S[j]$ into the B cells of its block, one after the other.



TD1 - PRAM

$$\text{BCASTP}(e, N, P)$$

```

1:  $B \leftarrow N/P$ 
2:  $S[1 \dots P] \leftarrow \text{BCAST}(e, P)$  ► (a)
3: forall  $j = 1, \dots, P$  in parallel do
4:   for  $i = (j-1)B + 1, \dots, jB$  do
5:      $pe[i] \leftarrow S[j]$  ► block  $j$ 
6: return  $pe[1 \dots N]$ 

```

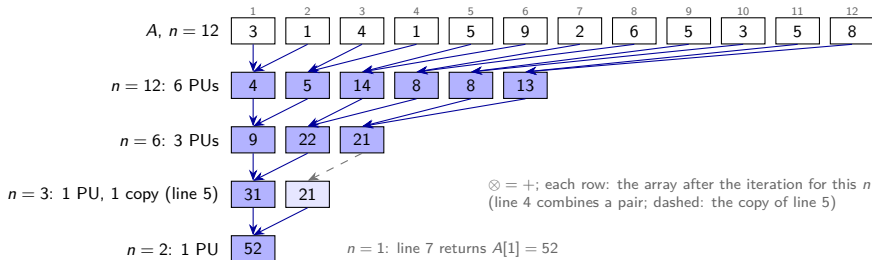
Exercise 4: Efficient broadcast and reduction

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

- a) The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name BCAST(e, N), so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- b) Write a *recursive* version BCASTREC(e, N) of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- c) Design BCASTP(e, N, P) that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- d) Write an *iterative* version REDUCE($A[1 \dots N], \otimes$) of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- e) The reduction of the lecture is *recursive*: write it as REDUCEREC($A[1 \dots N], \otimes$). Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- f) Design REDUCEP($A[1 \dots N], \otimes, P$) that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

université
PARIS-SACLAY

The halving of the lecture, written as a loop on the current size n of the array: combine the pairs, one PU per pair; an odd last element is copied. The sizes are 12, 6, 3, 2, 1, with 6, 3, 1, 1 PUs combining pairs.



TD1 - PRAM

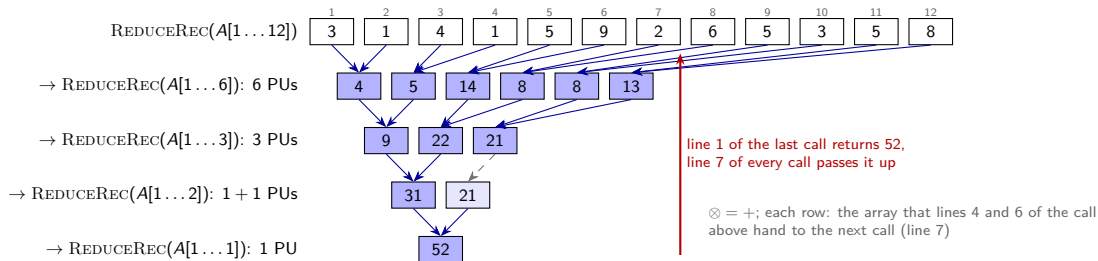
Exercise 4: Efficient broadcast and reduction

Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

- a) The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name BCAST(e, N), so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- b) Write a *recursive* version BCASTREC(e, N) of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- c) Design BCASTP(e, N, P) that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- d) Write an *iterative* version REDUCE($A[1 \dots N], \otimes$) of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- e) The reduction of the lecture is *recursive*: write it as REDUCEREC($A[1 \dots N], \otimes$). Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- f) Design REDUCEP($A[1 \dots N], \otimes, P$) that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

université
PARIS-SACLAY

The recursion of the lecture: each call combines its pairs and hands the half-size array to the next call; the result comes back up through the four returns. The same sizes and the same operations as the loop of (d).



Let e be a value, $A[1 \dots N]$ an array and $\otimes$ an associative operation. In the lecture, $\text{BCAST}(e, N)$ returned $pe[1 \dots N]$ with $pe[i] = e$ for all i , and $\text{REDUCE}(A[1 \dots N], \otimes)$ computed $A[1] \otimes \dots \otimes A[N]$, both in $O(\log_2 N)$ time on an EREW PRAM with N PUs, whereas $T_{seq}(N) = O(N)$ for both problems.

- a) The BCAST of the lecture is *iterative* (a loop on k , the number of doublings done so far) and creates N PUs at every iteration. Modify it, keeping the name BCAST(e, N), so that it uses up to N PUs, still runs in $O(\log_2 N)$ time, and becomes work-optimal. Is it efficient?
- b) Write a *recursive* version BCASTREC(e, N) of the broadcast that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- c) Design BCASTP(e, N, P) that does the same broadcast with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?
- d) Write an *iterative* version REDUCE($A[1 \dots N], \otimes$) of the reduction that uses up to N PUs, runs in $O(\log_2 N)$ time, and is work-optimal. Is it efficient?
- e) The reduction of the lecture is *recursive*: write it as REDUCEREC($A[1 \dots N], \otimes$). Is it work-optimal? Is it efficient? Which PUs does it create, compared with your iterative version?
- f) Design REDUCEP($A[1 \dots N], \otimes, P$) that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

TD1 - PRAM

- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array**
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

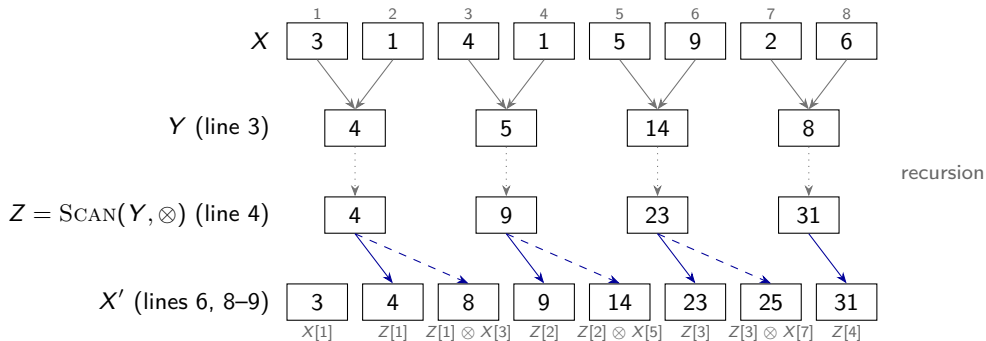
Exercise 5: Prefix scan on an array

Let $X[1 \dots N]$ be an array of size N and $\otimes$ an associative operation.

- a) Design an EREW PRAM algorithm $\text{SCAN}(X[1 \dots N], \otimes)$ that computes and returns the prefix scan array $X'[i] = X[1] \otimes \dots \otimes X[i]$ for all i , using N PUs and running in $O(\log_2 N)$ time. Verify that all reads and writes are exclusive. Hint: Try using recursion on the array. Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- b) This time, design an algorithm specifically for P PUs that does the same computation in $O(N/P + \log_2 P)$ time, then compute its work and cost. Is it work-optimal? For what values of P does it stay efficient?

$$\otimes = +$$

Combine the elements two by two into Y ; the recursive call returns Z , the prefix scan of Y , so $Z[k] = X[1] \otimes \cdots \otimes X[2k]$; X' is read off Z : the even positions are copied from Z , the odd ones add one more element of X .



Exercise 5(a): SCAN on an array, by recursion

- **Recursion on the array:** combine the elements two by two (line 3), scan the $\lfloor N/2 \rfloor$ pairs recursively (line 4), then deduce the prefixes of the original array.
- If Z is the prefix scan of the pairs, $Z[k] = X[1] \otimes \cdots \otimes X[2k]$: the prefix of an even index $2k$ is $Z[k]$ (line 6), the prefix of an odd index $2k + 1$ is $Z[k] \otimes X[2k + 1]$ (line 9).
- **Exclusivity:** line 3, PU i reads $X[2i - 1]$, $X[2i]$ and writes $Y[i]$; line 6, PU i reads $Z[i]$ and writes $X'[2i]$; line 9, PU i reads $Z[i - 1]$, $X[2i - 1]$ and writes $X'[2i - 1]$: all different from one PU to another.
- $Z[k]$ is needed by two indices, $2k$ and $2k + 1$: the two loops (lines 5 and 7) are kept *separate*, so that each cell is read once per loop. **EREW.**

$$\text{SCAN}(X[1 \dots M], \otimes)$$

```

1: if  $N = 1$  then return  $X[1 \dots 1]$ 
2: forall  $i = 1, \dots, \lfloor N/2 \rfloor$  in parallel do
3:    $Y[i] \leftarrow X[2i - 1] \otimes X[2i]$ 
4:  $Z[1 \dots \lfloor N/2 \rfloor] \leftarrow \text{SCAN}(Y[1 \dots \lfloor N/2 \rfloor], \otimes)$ 
5: forall  $i = 1, \dots, \lfloor N/2 \rfloor$  in parallel do
6:    $X'[2i] \leftarrow Z[i]$ 
7: forall  $i = 1, \dots, \lceil N/2 \rceil$  in parallel do
8:   if  $i = 1$  then  $X'[1] \leftarrow X[1]$ 
9:   else  $X'[2i - 1] \leftarrow Z[i - 1] \otimes X[2i - 1]$ 
10: return  $X'[1 \dots N]$ 

```

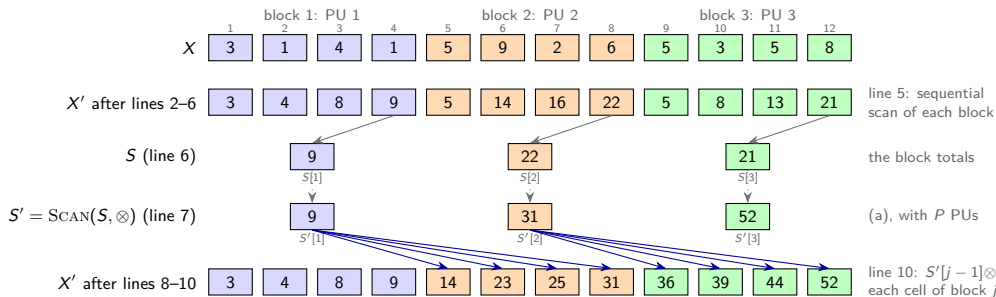

Exercise 5: Prefix scan on an array

Let $X[1 \dots N]$ be an array of size N and $\otimes$ an associative operation.

- a) Design an EREW PRAM algorithm $\text{SCAN}(X[1 \dots N], \otimes)$ that computes and returns the prefix scan array $X'[i] = X[1] \otimes \dots \otimes X[i]$ for all i , using N PUs and running in $O(\log_2 N)$ time. Verify that all reads and writes are exclusive. Hint: Try using recursion on the array. Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- b) This time, design an algorithm specifically for P PUs that does the same computation in $O(N/P + \log_2 P)$ time, then compute its work and cost. Is it work-optimal? For what values of P does it stay efficient?

université
PARIS-SACLAY

The recipe of exercise 4: P blocks of $B = N/P$ elements. Every PU scans its own block sequentially and records its total $S[j]$; the P totals are scanned in parallel with (a); every PU then applies $S'[j - 1]$, the combination of all the elements before its block, to its block.



Exercise 5(b): SCANP, prefix scan with P PUs

- P blocks of $B = N/P$ consecutive elements (P divides N ; otherwise the last block is shorter).
- Three phases: every PU scans its own block *sequentially* (lines 2–6) and records its total $S[j]$; the P totals are scanned *in parallel* with (a) (line 7); every PU then applies $S'[j - 1]$, the combination of all the elements *before* its block, to its block (lines 8–10).
- Order of the operands in line 10: $S'[j - 1]$ goes on the *left*, as these elements come first ($\otimes$ need not be commutative).
- Exclusivity: in lines 2–6 and 8–10, PU j touches only the cells of its block and $S[j]$, $S'[j - 1]$; line 7 is the EREW algorithm of (a). **EREW**.

$$\text{SCANP}(X[1 \dots N], \otimes, P)$$

```

1:  $B \leftarrow N/P$ 
2: forall  $j = 1, \dots, P$  in parallel do
3:    $X'[(j-1)B + 1] \leftarrow X[(j-1)B + 1]$ 
4:   for  $i = (j-1)B + 2, \dots, jB$  do
5:      $X'[i] \leftarrow X'[i-1] \otimes X[i]$   $\blacktriangleright$  scan of block  $j$ 
6:    $S[j] \leftarrow X'[jB]$   $\blacktriangleright$  total of block  $j$ 
7:  $S'[1 \dots P] \leftarrow \text{SCAN}(S[1 \dots P], \otimes)$   $\blacktriangleright$  (a)
8: forall  $j = 2, \dots, P$  in parallel do
9:   for  $i = (j-1)B + 1, \dots, jB$  do
10:     $X'[i] \leftarrow S'[j-1] \otimes X'[i]$ 
11: return  $X'[1 \dots N]$ 

```


- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list**
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Exercise 6: Scan on a list

Let $next[1 \dots N]$ be a linked list of N nodes, with head h and tail t ($next[t] = \text{NIL}$), $x[1 \dots N]$ a value attached to each node, and $\otimes$ an associative operation. In the lecture, the list ranking algorithm computed, with pointer jumping, the distance $d[i]$ from every node i to the end of the list.

- Design an algorithm $\text{SCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$ that computes and returns the array $y[1 \dots N]$ where $y[i] = x[h] \otimes \dots \otimes x[i]$ is the combination of the values from the head h of the list to node i , in $O(\log_2 N)$ time using N PUs. Hint: start from the list ranking algorithm of the lecture.
- Verify the memory accesses of your algorithm: are all reads exclusive? all writes? On which PRAM model does it run as written, and how would you modify it to run on an EREW PRAM?
- Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- Show that computing the rank $r[i]$ of every node from the head of the list (0 for the head, $N - 1$ for the tail) is a particular case of SCAN : for which values $x[i]$ and which operation $\otimes$?
- Using a single call to SCAN , design an algorithm $\text{RANKBYPREFIX}(\text{next}[1 \dots N])$ with N PUs that computes and returns the distance $d[i]$ from every node to the end of the list.
- Design an algorithm $\text{REVERSESCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$ that computes, in the same time, the array $y[1 \dots N]$ where $y[i] = x[i] \otimes \dots \otimes x[t]$ is the combination of the values from node i to the tail t of the list. Pay attention to the order of the operands if $\otimes$ is not commutative.

$\otimes$ is the concatenation (abc stands for $a \otimes b \otimes c$): after k iterations, $y[i]$ combines the values of the 2^k nodes ending at i (fewer near the head), and $prev[i]$ points 2^k nodes back.



Exercise 6(a): SCAN on a list, by pointer jumping

- For the prefix from the head, **jump towards the head**: $prev \leftarrow \text{INVERT}(next)$ (exercise 3, line 1), then the pointer jumping of RANKCOMPUTATION along $prev$, each node **accumulating the value of the node it jumps over** (line 7).
- Invariant: $y[i]$ is the combination of the values from the node just after $prev[i]$ to i (from h if $prev[i] = \text{NIL}$). True after line 3; line 7 preserves it: $y[prev[i]] \otimes y[i]$ covers the nodes from the node after $prev[prev[i]]$ to i .
- Order of the operands: the node jumped over is *before* i , its value goes on the *left* ($\otimes$ need not be commutative).
- After $\lceil \log_2 N \rceil$ iterations every $prev[i]$ is NIL , and $y[i] = x[h] \otimes \dots \otimes x[i]$. The list $next$ is left untouched.

$$\text{SCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$$

```

1:  $prev[1 \dots N] \leftarrow \text{INVERT}(next[1 \dots N])$ 
2: forall  $i = 1, \dots, N$  in parallel do
3:    $y[i] \leftarrow x[i]$ 
4:   while  $\exists i$  with  $prev[i] \neq \text{NIL}$  do
5:     forall  $i = 1, \dots, N$  in parallel do
6:       if  $prev[i] \neq \text{NIL}$  then
7:          $y[i] \leftarrow y[prev[i]] \otimes y[i]$ 
8:          $prev[i] \leftarrow prev[prev[i]]$ 
9: return  $y[1 \dots N]$ 

```


TD1 - PRAM

Exercise 6: Scan on a list

Let $next[1 \dots N]$ be a linked list of N nodes, with head h and tail t ($next[t] = \text{NIL}$), $x[1 \dots N]$ a value attached to each node, and $\otimes$ an associative operation. In the lecture, the list ranking algorithm computed, with pointer jumping, the distance $d[i]$ from every node i to the end of the list.

- a) Design an algorithm $\text{SCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$ that computes and returns the array $y[1 \dots N]$ where $y[i] = x[h] \otimes \dots \otimes x[i]$ is the combination of the values from the head h of the list to node i , in $O(\log_2 N)$ time using N PUs. Hint: start from the list ranking algorithm of the lecture.
- b) Verify the memory accesses of your algorithm: are all reads exclusive? all writes? On which PRAM model does it run as written, and how would you modify it to run on an EREW PRAM?
- c) Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- d) Show that computing the rank $r[i]$ of every node from the head of the list (0 for the head, $N - 1$ for the tail) is a particular case of SCAN: for which values $x[i]$ and which operation $\otimes$?
- e) Using a single call to SCAN, design an algorithm $\text{RANKBYPREFIX}(\text{next}[1 \dots N])$ with N PUs that computes and returns the distance $d[i]$ from every node to the end of the list.
- f) Design an algorithm $\text{REVERSESCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$ that computes, in the same time, the array $y[1 \dots N]$ where $y[i] = x[i] \otimes \dots \otimes x[t]$ is the combination of the values from node i to the tail t of the list. Pay attention to the order of the operands if $\otimes$ is not commutative.

- Towards the tail, the jumps follow the list itself: **no inversion**, and the node jumped over is now *after i*: its value goes on the *right* (line 7).
- Invariant: $y[i]$ is the combination of the values from i to the node just before $cnext[i]$ (to t if $cnext[i] = \text{NIL}$); after $\lceil \log_2 N \rceil$ iterations, $y[i] = x[i] \otimes \cdots \otimes x[t]$: exactly RANKCOMPUTATION with $\otimes$ instead of $+$.
- The jumps overwrite the pointers they follow (line 8): they are made on a **copy cnext** of the list (line 2), so that *next* is left untouched, as SCAN left it by jumping on *prev*.
- Same time, work, cost and models as SCAN, with the same termination test as in (b).
- With $\otimes = +$ and $x[i] = 1$ ($x[t] = 0$): REVERSESCAN computes $d[i]$ directly, no subtraction; it *is* RANKCOMPUTATION.

$$\text{REVERSESCAN}(next, x, \otimes)$$

```

1: forall  $i = 1, \dots, N$  in parallel do
2:    $cnext[i] \leftarrow next[i]$     ► jumps on the copy
3:    $y[i] \leftarrow x[i]$ 
4:   while  $\exists i$  with  $cnext[i] \neq \text{NIL}$  do
5:     forall  $i = 1, \dots, N$  in parallel do
6:       if  $cnext[i] \neq \text{NIL}$  then
7:          $y[i] \leftarrow y[i] \otimes y[cnext[i]]$ 
8:          $cnext[i] \leftarrow cnext[cnext[i]]$ 
9:   return  $y[1 \dots N]$ 

```

- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM**
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Let $next[1 \dots N]$ be a linked list of N nodes, each node i having a label $label[i] \in \{\text{red}, \text{blue}\}$.

- a) Design an EREW PRAM algorithm $\text{SELECTBLUES}(next[1 \dots N], label[1 \dots N])$ with N PUs that updates the list $next[1 \dots N]$ so that in the end i) each blue node points to the next blue node in the original list, ii) the last blue node and all the red nodes point to NIL, and iii) the algorithm returns the index of the first blue node, the head of the updated list of blue nodes (NIL if there is no blue node).
- b) What are its time, work and cost?

- Every node must find the **first blue node at or after itself**: a REVERSESCAN (6(f)) with a well-chosen operation.
- Mark: $x[i] = i$ if i is blue, NIL otherwise (line 2). Combine with $a \otimes b = a$ if $a \neq \text{NIL}$, b otherwise (“first non-NIL value”, associative): $y[i]$ is the first blue node from i included, or NIL (line 3).
- Relink: a blue node i points to $y[\text{next}[i]]$ (line 7), NIL for the last blue node; red nodes to NIL (line 9); the head of the blue list is $y[h]$ (h : exercise 2), NIL if no blue node.
- REVERSESCAN jumps on its own copy of the list: next is intact for line 7.
- Exclusivity: line 7 reads $y[\text{next}[i]]$, next injective; other accesses on own cells; REVERSESCAN with the test of 6(b). **EREW**.

```

1: forall  $i = 1, \dots, N$  in parallel do
2:   if  $label[i] = \text{blue}$  then  $x[i] \leftarrow i$  else  $\text{NIL}$ 
3:  $y[1 \dots N] \leftarrow \text{REVERSESCAN}(next, x, \otimes)$ 
4:  $h \leftarrow \text{head of the list}$  ► exercise 2
5: forall  $i = 1, \dots, N$  in parallel do
6:   if  $label[i] = \text{blue}$  and  $next[i] \neq \text{NIL}$  then
7:      $next[i] \leftarrow y[next[i]]$ 
8:   else
9:      $next[i] \leftarrow \text{NIL}$ 
10: return  $y[h]$ 

```


- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM**
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication

Exercise 8: Separation in an array on an EREW PRAM

Let $A[1 \dots N]$ be an array of N objects, each element having a label $\text{LABEL}(A[i]) \in \{0, 1\}$.

- a) Design an EREW PRAM algorithm $\text{SEPARATE01}(A[1 \dots N])$ with N PUs that returns a copy $A'[1 \dots N]$ of A permuted so that all the elements of label 0 come first (the labels of $A'[1], \dots, A'[N]$ read $0, \dots, 0, 1, \dots, 1$). Hint: Try to use the prefix scan algorithm to determine where to write each element after permutation.
- b) What are its time, work and cost?

- SEPARATE01(
- $A[1 \dots N]$
-)

```

1: forall  $i = 1, \dots, N$  in parallel do
2:   if LABEL( $A[i]$ ) = 0 then  $z[i] \leftarrow 1$  else  $z[i] \leftarrow 0$ 
3:  $Z[1 \dots N] \leftarrow \text{SCAN}(z[1 \dots N], +)$  ► exercise 5
4:  $pn_0[1 \dots N] \leftarrow \text{BCAST}(Z[N], N)$ 
5: forall  $i = 1, \dots, N$  in parallel do
6:   if LABEL( $A[i]$ ) = 0 then  $A'[Z[i]] \leftarrow A[i]$ 
7:   else  $A'[pn_0[i] + i - Z[i]] \leftarrow A[i]$ 
8: return  $A'[1 \dots N]$ 

```


- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers**
- 10 Exercise 10: Parallel matrix multiplication

Exercise 9: Sorting an array of unique integers

Let $X[1 \dots N]$ be an array of unique integers, meaning no two elements $X[i]$ and $X[j]$ are equal for $1 \leq i \neq j \leq N$.

- Design a CRCW PRAM algorithm $\text{SORT}(X[1 \dots N])$ with N^2 PUs, running in $O(1)$ time, that returns the sorted array $X'[1 \dots N]$, such that $X'[i] < X'[i + 1]$ for all i . Specify the concurrency mode used for the concurrent writes.
- What are its time, work and cost?
- How can you make the algorithm work if the elements of $X[1 \dots N]$ are not unique?
- How can you make the algorithm work on a CREW PRAM, still with N^2 PUs? What are its time, work and cost?

TD1 - PRAM

TD1 - PRAM

- 1 Exercise 1: Is the array sorted?
- 2 Exercise 2: From a list to a ring
- 3 Exercise 3: Inverting a list
- 4 Exercise 4: Efficient broadcast and reduction
- 5 Exercise 5: Prefix scan on an array
- 6 Exercise 6: Scan on a list
- 7 Exercise 7: Selection in a list on an EREW PRAM
- 8 Exercise 8: Separation in an array on an EREW PRAM
- 9 Exercise 9: Sorting an array of unique integers
- 10 Exercise 10: Parallel matrix multiplication**

Let $A[1 \dots N][1 \dots N]$ and $B[1 \dots N][1 \dots N]$ be two square matrices. Their product is the square matrix $C = AB$ whose elements are defined as follows:

$$C[i][j] = \sum_{k=1}^N A[i][k] B[k][j], \quad 1 \leq i, j \leq N$$

- Design a CRCW PRAM algorithm $\text{MATMUL}(A, B)$ with N^3 PUs, running in $O(1)$ time, that carries out the multiplication of two $N \times N$ matrices $C = AB$, then returns the matrix C . What are its time, work and cost?
- Do the same for a CREW PRAM, still with N^3 PUs. What are its time, work and cost? How can you make this algorithm work on an EREW PRAM as well?

Exercise 10(a): MATMUL on a CRCW PRAM

- **One PU per product** $A[i][k]B[k][j]$: N^3 PUs, indexed by (i, j, k) , and the N products of each $C[i][j]$ are summed by a concurrent write.
- Line 4: **fusion mode with +**: the N PUs $(i, j, 1), \dots, (i, j, N)$ write the cell $C[i][j]$, which receives the sum of the written values (initialized to 0 in line 2).
- The reads are concurrent as well: $A[i][k]$ is read by the N PUs $(i, 1, k), \dots, (i, N, k)$, and $B[k][j]$ by the N PUs $(1, j, k), \dots, (N, j, k)$.
- The same trick as SUMARRAYCRCW: the $N - 1$ additions of each $C[i][j]$ are done by the memory.

$$\text{MATMUL}(A, B)$$

```

1: forall  $(i, j) \in \{1, \dots, N\}^2$  in parallel do
2:    $C[i][j] \leftarrow 0$ 
3: forall  $(i, j, k) \in \{1, \dots, N\}^3$  in parallel do
4:    $C[i][j] \leftarrow A[i][k] \cdot B[k][j]$  ► fusion +
5: return  $C$ 

```

- Time $O(1)$: lines 1–2 and 3–4 are two parallel loops of $O(1)$.
- N^3 PUs, one per product: with fewer PUs, some PU would do several products one after the other.
- Work $O(N^3)$ (one product per PU, all created), cost $O(N^3)$.
- With respect to the standard sequential algorithm, $T_{seq}(N) = O(N^3)$: **work-optimal** and **efficient** (faster sequential algorithms exist, such as Strassen's, which we ignore here).
- Too good to be true, as SUMARRAYCRCW: the additions are done by the memory, for free.

```

1: forall  $(i, j) \in \{1, \dots, N\}^2$  in parallel do
2:    $C[i][j] \leftarrow 0$ 
3: forall  $(i, j, k) \in \{1, \dots, N\}^3$  in parallel do
4:    $C[i][j] \leftarrow A[i][k] \cdot B[k][j]$  ► fusion +
5: return  $C$ 

```

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ≡ ↺ 🔍 ↻

Let $A[1 \dots N][1 \dots N]$ and $B[1 \dots N][1 \dots N]$ be two square matrices. Their product is the square matrix $C = AB$ whose elements are defined as follows:

$$C[i][j] = \sum_{k=1}^N A[i][k] B[k][j], \quad 1 \leq i, j \leq N$$

- Design a CRCW PRAM algorithm $\text{MATMUL}(A, B)$ with N^3 PUs, running in $O(1)$ time, that carries out the multiplication of two $N \times N$ matrices $C = AB$, then returns the matrix C . What are its time, work and cost?
- Do the same for a CREW PRAM, still with N^3 PUs. What are its time, work and cost? How can you make this algorithm work on an EREW PRAM as well?

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Paris, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com